Seguridad en Redes y Análisis Inteligente de Amenazas

Tema 6. Criptografía aplicada a la seguridad en redes

Índice

Esquema

Ideas clave

6.1. Introducción y objetivos

6.2. Cifrado simétrico

6.3. Funciones resumen hash

6.4. Cifrado asimétrico

6.5. Referencias bibliográficas

A fondo

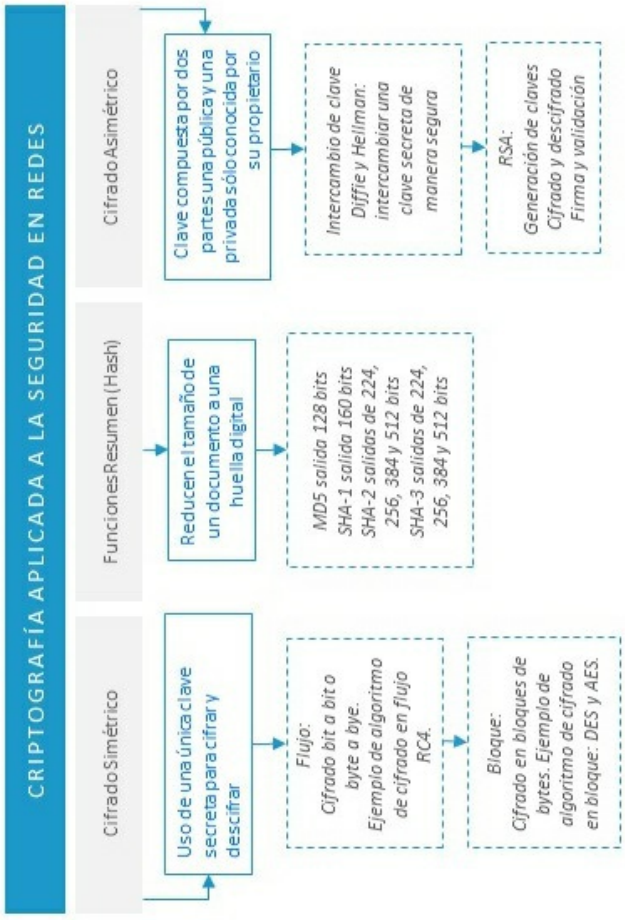
Libro Electrónico de Seguridad Informática y Criptografía

Capítulos 5, 9 y 11 del libro Cryptography and Network Security: Principles and Practice

Canal de YouTube <computerphile>

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

6.1. Introducción y objetivos

En este tema profundizaremos en la **criptografía aplicada,** comenzando por la cifra simétrica, pasando por las funciones resumen (*hash*) y finalizando con la cifra asimétrica. Entre los algoritmos a estudiar se encuentran RC4, AES, MD5, SHA-2, el intercambio de Diffie y Hellman y RSA.

En la **cifra simétrica,** en flujo el texto en claro se cifrará bit a bit, o byte a byte, con una secuencia de bits o bytes que forman la clave, conocida **como secuencia de clave Si.** No se trata de una clave cuyo valor sea único y que se use durante todo el proceso de cifra, como se sucederá en la cifra en bloques, sino que esta va cambiando a medida que se recorre el texto en claro. Por su parte, en la cifra en bloque se usará una única clave, que comparten emisor y receptor y que no cambiará durante el proceso de cifra. El emisor que cifra el mensaje formará bloques de tamaños típicos entre 64 y 128 bits, existiendo diferentes modos de cifra.

Posteriormente, se analizarán las características y propiedades de las funciones *hash*, así como su utilidad en la criptografía; para terminar, se estudiarán las funciones *hash* más populares, como son MD5, SHA-1, SHA-256 y SHA-3.

Por último, se presentan las características de la cifra asimétrica o por clave pública, se estudiará el intercambio de clave de Diffie y Hellman, así como el algoritmo RSA. La principal característica de la cifra asimétrica reside en el hecho de que la clave, que hasta ahora era única y secreta, se divide en dos partes, donde una de ellas se mantiene en secreto, pero la otra es pública.

Los **objetivos** del tema se resumen a continuación:

- Conocer los principios básicos de la cifra simétrica y algunos de sus algoritmos fundamentales.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Identificar las características y propiedades de las funciones *hash,* así como el funcionamiento de MD5, SHA-1 y SHA-2.
- Conocer los principios básicos de la cifra simétrica y algunos de sus algoritmos fundamentales: Diffie y Hellman y RSA.

© Universidad Internacional de La Rioja (UNIR)

6.2. Cifrado simétrico

Cifra en flujo

La Figura 1 muestra el esquema de un sistema de cifra en flujo.

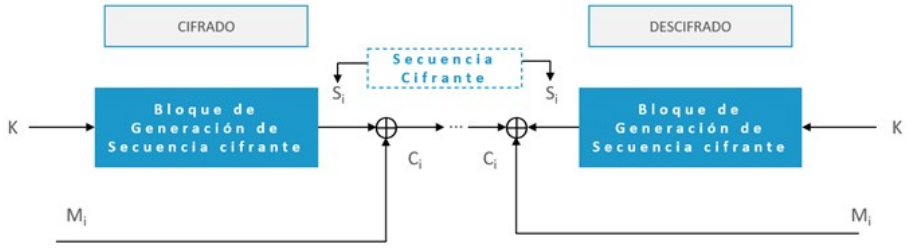


Figura 1. Esquema de la cifra en flujo. Elaboración propia.

Siguiendo el esquema de la Figura 1, estos sistemas de cifra se resumen en las siguientes tres **características:**

- Un algoritmo de cifrado y descifrado con la función XOR.
- Una secuencia cifrante binaria y pseudoaleatoria Si que se obtiene a partir de una clave secreta K y un algoritmo generador de esa secuencia.
- Una misma operación de cifrado que de descifrado debido el carácter involutivo de la función XOR.

Algoritmo de cifra en flujo, RC4

En cuanto a RC4, es el acrónimo de Ron's Code 4, un algoritmo de cifra en flujo byte a byte diseñado por Ron Rivest en 1987. Su uso más habitual se encontraba hasta el año 2014 en el protocolo SSL/TLS, especialmente al utilizar Internet Explorer, aunque se usa también en PEP Wired Equivalent Privacy y en WPA Wi-Fi Protected Access, para el cifrado en redes inalámbricas.

El algoritmo consta dos rutinas: KSA, con la que se genera la clave; y PRGA, con la

© Universidad Internacional de La Rioja (UNIR)

que se cifra cada byte del texto en claro con un nuevo byte de la secuencia de clave, y que se muestran en la Figura 2.

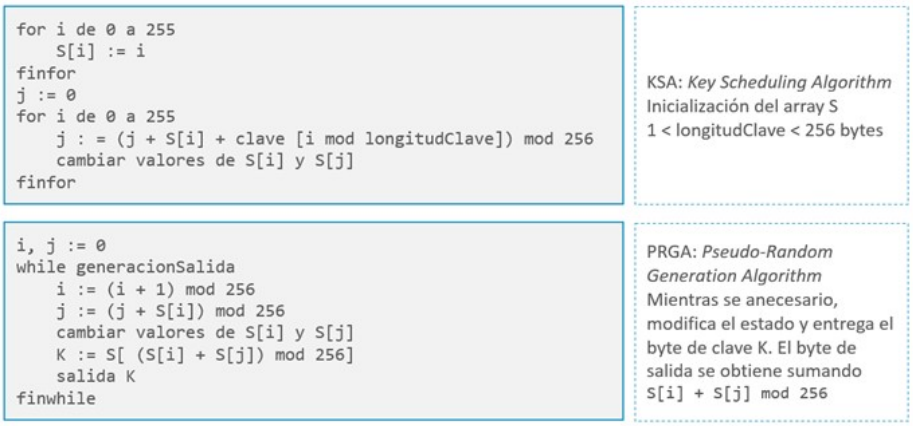


Figura 2. Rutinas KSA y PRGA de RC4. Elaboración propia.

Su popularidad estaba basada fundamentalmente en su alta velocidad y la simplicidad de diseño, tanto *hardware* como *software*, y es casi el doble de rápido que AES, el actual estándar de cifra simétrica.

Cifra en bloque

En la cifra simétrica en bloque, la información a cifrar o texto en claro se agrupa en bloques *n* de bits, típicamente 128 bits o 16 octetos. Estos bloques de bits entran al cifrador y se mezclan con los bits clave, mediante diversas operaciones tanto de permutación como de sustitución; por esto también son conocidos como **cifradores de producto.**

Cada bloque de texto en claro se convierte en un bloque de texto cifrado, como se muestra en la Figura 3. Al final del texto en claro se ha añadido un relleno por si este no es congruente con el tamaño del bloque.

© Universidad Internacional de La Rioja (UNIR)

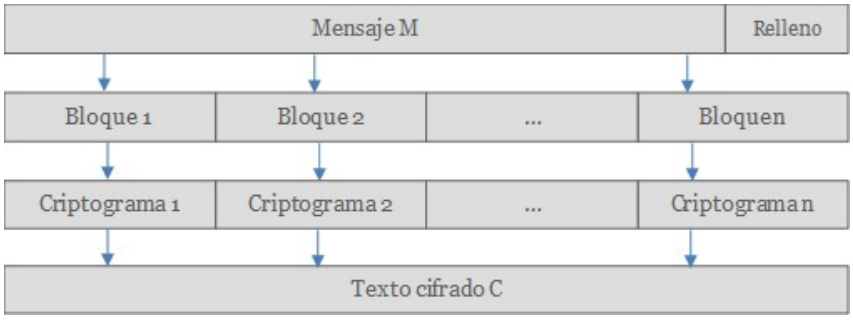


Figura 3. Bloques en la cifra simétrica. Fuente: Ramió, 2006.

Para que los bits del texto en claro se mezclen lo suficiente con los bits de la clave, habrá que aplicar permutaciones y sustituciones en esos bits varias veces. En criptografía, a estas operaciones repetitivas se les llama **vueltas o rondas del algoritmo.** Con ello se logra, entre otras características interesantes, el denominado **efecto de avalancha;** esto es, que el cambio de un solo bit en la entrada (o en la clave) produzca un cambio en aproximadamente el 50 % de los bits de salida. En otras palabras, que cada bit de salida sea una función compleja que dependa en un 50 % de cada bit de entrada.

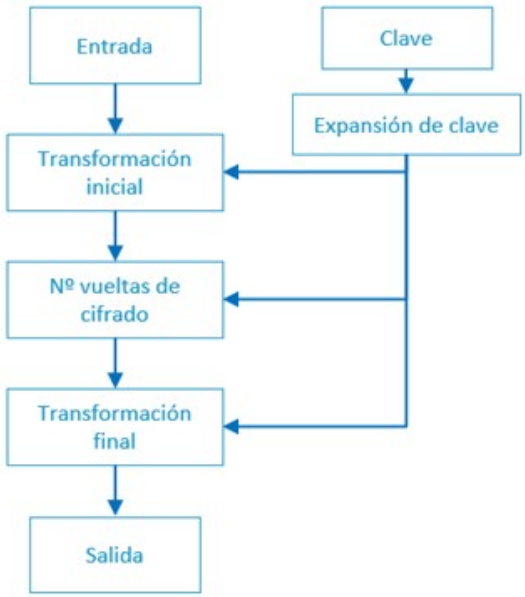


Figura 4. Esquema de la cifra simétrica. Elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

La Figura 4 muestra el esquema de un cifrado simétrico y las operaciones características que se realizan en cada fase del algoritmo. Cada bloque se cifra durante varias vueltas en las que se usa una subclave, que se calcula a partir de la clave de cifra K con una función que se conoce como **expansión de clave,** de forma que en cada vuelta esta sea distinta.

Modos de cifra en bloque

Aunque existe una media docena modos de cifra, los más conocidos son: ECB (*Electronic CodeBook*), CBC (*Cipher Block Chaining*) y CTR (*Counter*).

Modo ECB

El modo de **cifra por libro electrónico de códigos ECB** (*Electronic Codebook*) consiste en cifrar bloques de texto en claro de manera independiente. Así, cada bloque de *x* bits del texto en claro se cifra con la clave *K* del algoritmo y entrega un bloque de texto cifrado de *x* bits.

No se recomienda el uso de este modo porque es vulnerable a ataques por inicios y finales iguales entre dos documentos diferentes, así como a ataques por repetición de bloques elegidos o bien modificados.

Modo CBC

En el **modo de cifra por encadenamiento de bloques cifrantes** (*Cipher Block Chaining*) CBC, se usa una segunda clave de igual tamaño que el bloque de texto a cifrar, llamada **vector inicial IV,** con la cual se realiza un OR exclusivo con el primer bloque del texto en claro a cifrar, antes de comenzar la cifra.

El resultado de la cifra de esta nueva entrada con la clave *K*, es decir C1, se usará como vector inicial del siguiente bloque de texto a cifrar, y así sucesivamente. Por tanto, se va encadenando el cifrado anterior con el nuevo cifrado, usándolo como vector de suma OR exclusivo para el nuevo bloque de texto a cifrar.

© Universidad Internacional de La Rioja (UNIR)

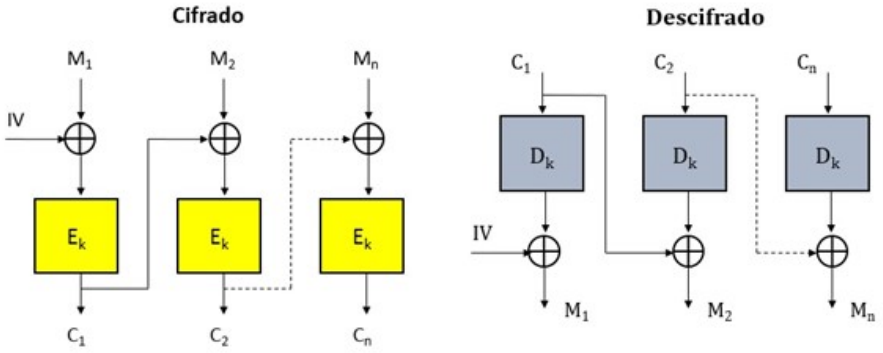


Figura 5. Modo de cifra *CBC Cipher Block Chaining*. Fuente: Ramió, 2006.

Para descifrar con este método, una vez se ha aplicado el algoritmo de descifrado al primer subcriptograma con la clave *K*, se recupera el texto en claro realizando una suma OR exclusivo con el vector inicial IV. Los siguientes bloques de subcriptogramas se descifran de igual manera, pero utilizando ahora como vector inicial el criptograma anterior, que hemos guardado previamente en un registro temporal.

Un error en la transmisión utilizando el modo CBC afectará a todo el resto del criptograma. Unido a esto, aparece el problema asociado de que CBC no permite paralelizar la operación de descifrado, algo a tener en cuenta cuando hablamos de cifrado de grandes volúmenes de información.

Modo CTR

El **modo CTR** es similar al CBC, pero soluciona el problema del no paralelismo que tiene CBC. Además, a diferencia del modo CBC un error de transmisión afectará solo al bloque en cuestión, no a los siguientes.

CTR usa ese mismo vector inicial IV, pero en modo contador y no sumándolo XOR al texto en claro, sino como entrada al propio algoritmo de cifra, de forma que para el primer bloque su valor es IV, para el segundo bloque es IV+1, etc. La salida de esa cifra se suma OR exclusivo con el bloque de texto en claro a cifrar.

© Universidad Internacional de La Rioja (UNIR)

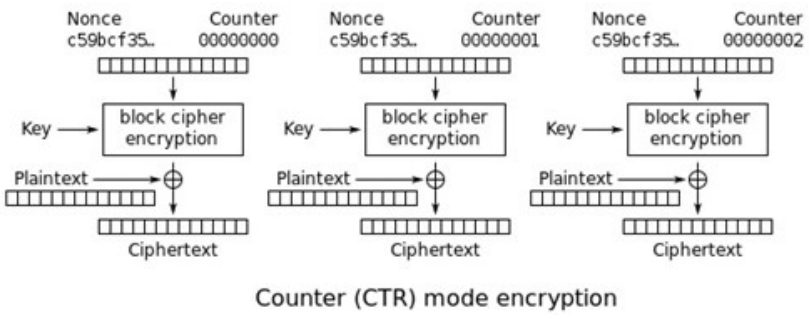


Figura 6. Modo de cifra CTR *Counter*. Fuente: Wikipedia.

Algoritmo de cifra en bloque, AES

El **algoritmo AES** (*Advanced Encryption Standard*) nace como el sustituto del DES como estándar mundial de cifra simétrica. En noviembre de 2001, el NIST anuncia oficialmente a través del Federal Information Processing Standard Publications, FIPS 197, que el nuevo estándar para cifrado simétrico del siglo XXI será el AES.

AES es un algoritmo no de tipo Feistel, que procesa bloques completos de texto en claro de 128 bits, con claves estándar de 128, 192 o 256 bits. Para ello, usa una matriz de estado de tamaño 4x4, cuyas 16 celdas o bytes van cambiando de valor de acuerdo con los procesos que ejecuta el algoritmo. En el cifrado se utilizan técnicas de sustitución y permutación, en algunos casos con operaciones polinómicas dentro de un cuerpo. Todas las operaciones de cifra dentro de la matriz de estado se realizan sobre bytes, en palabras de 32 bits que se escriben de arriba hacia abajo y de izquierda a derecha.

Las **cuatro operaciones básicas** de cifrado en el AES son AddRoundKey, SubBytes, ShiftRows y MixColumns (ver Figura 7).

- La **operación AddRoundKey** ejecuta la suma OR exclusivo entre los bytes del mensaje y los bytes de la clave.
- La **operación SubBytes** ejecuta una sustitución de cada uno de los 16 bytes de la matriz de estado mediante una tabla.

© Universidad Internacional de La Rioja (UNIR)

- La **operación ShiftRows** consiste en una permutación de las filas del estado de forma que la primera fila no rota, la segunda rotará un byte, la tercera rotará 2 bytes y la cuarta rotará 3 bytes.
- La **operación MixColumns** es algo más compleja y consiste en multiplicar cada una de las columnas de la matriz de estado por una matriz polinómica.

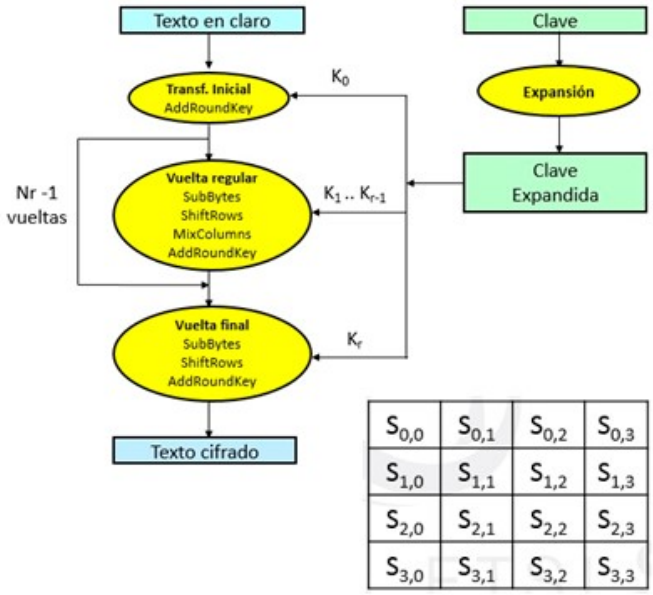


Figura 7. Esquema del algoritmo AES y matriz de estado. Fuente: Ramió, 2006.

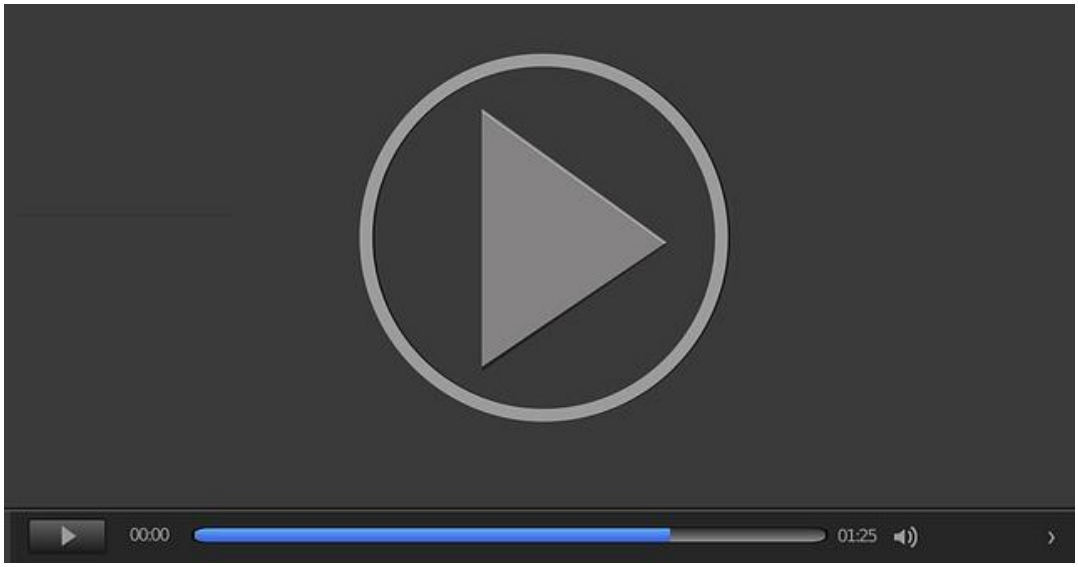
Aunque las operaciones que realizan estas funciones pueden representarse por tablas o figuras, de muy fácil comprensión, lo cierto es que están soportadas por ecuaciones polinómicas. Es decir, el algoritmo AES tiene un importante fondo matemático.

En el vídeo, *Cifrado simétrico utilizando software educativo*, se muestran ejemplos de cifrado y descifrado utilizando el algoritmo AES como ejemplo de cifrado en bloque.



© Universidad Internacional de La Rioja (UNIR)

Ideas clave



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=f0059634-d9ba-4381-8c4c-af63012d0df1

© Universidad Internacional de La Rioja (UNIR)

6.3. Funciones resumen hash

Las **funciones *hash*** son algoritmos que al aplicarlos sobre un mensaje, archivo o texto M, entregan un resumen de x bits conocido como h(M). Se trata de un número que a modo de huella digital representará a dichos documentos de forma supuestamente única. A diferencia de la popular familia de aplicaciones de tipo zip para la compresión de archivos, las funciones *hash* entregan siempre un resumen de un número fijo de bits, independientemente del tamaño de ese archivo. Es importante recalcar que, en tanto carecen de claves, las funciones *hash* no pueden considerarse propiamente como algoritmos de cifra.

Las funciones *hash* **juegan dos papeles muy importantes en la criptografía:**

Como **reducen el tamaño de un documento a una huella digital** de un conjunto pequeño de bits, normalmente unas centenas de bits, permiten adaptar ese documento para su firma digital, en tanto los algoritmos de clave pública que se usan para la firma digital son muy lentos (Kbytes/s) y no sería por tanto eficiente firmar documentos de miles o millones de bytes.

Como una de las propiedades de estas funciones es que, si se altera algún bit del mensaje, accidentalmente durante el tránsito o bien de forma deliberada, el resumen de ese otro mensaje será distinto, permitirá comprobar la integridad del mensaje.

Adicionalmente, hay muchas aplicaciones que requieren contraseñas, como pueden ser los sistemas de foros o bien servidores de correo, donde lo que se almacena en el servidor no es la contraseña del usuario sino el *hash* de esa contraseña. Esto último no se hace con el propósito de comprimir la contraseña, que con toda seguridad será más corta que el *hash* generado, sino con el propósito de que, si alguien tiene acceso al almacén de contraseñas, no pueda ver la contraseña como texto en claro y, por tanto, no pueda tener acceso al sistema, a no ser que rompa esa

© Universidad Internacional de La Rioja (UNIR)

contraseña por fuerza bruta u otro tipo de ataque si conoce qué función *hash* se ha empleado.

Función *hash* MD5

Diseñada en 1992 por Ron Rivest, esta función resumen está obsoleta desde mediados de 2005 al detectarse graves fallos en su diseño y, además, al ser su resumen de solo 128 bits. No obstante, se ha seguido utilizando hasta bien entrada la década de 2010, especialmente en aplicaciones para comprobar la integridad de archivos, acceso a cuentas de correo, así como en redes P2P.

MD5 procesa los mensajes en bloques de 512 bits y produce una salida de 128 bits. En primer lugar, expande el mensaje M hasta una longitud exactamente 64 bits inferior a un múltiplo de 512 bits. Para ello, añade como relleno un bit uno seguido de tantos bits ceros como sean necesarios, reservando los últimos 64 bits para indicar la longitud del mensaje (ver Figura 9).

El algoritmo tiene cuatro vectores iniciales ABCD, palabras de 32 bits cada uno, con los siguientes valores en hexadecimal: A = 01234567, B = 89ABCDEF, C = FEDCBA98, D = 76543210. A los vectores ABCD, que no son secretos, y al primer bloque de 512 bits se le aplicarán 64 operaciones de 32 bits obteniéndose cuatro nuevos vectores ABCD que serán los vectores de entrada para el siguiente bloque de 512 bits. Esta acción se repite con los restantes bloques del documento, encadenando los sucesivos vectores ABCD. El último bloque ABCD entregará el resumen final h(M) de 128 bits del mensaje M.

© Universidad Internacional de La Rioja (UNIR)

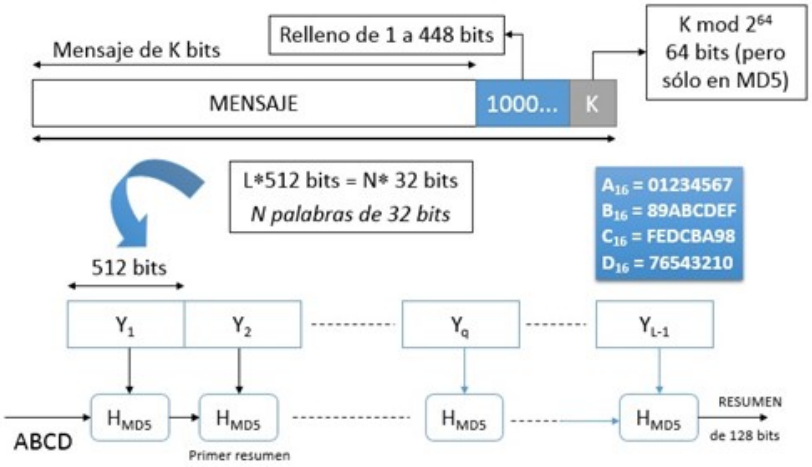


Figura 9. Formación de bloques en MD5 y esquema del *hash*. Fuente: Ramió, 2006.

MD5 realiza para cada bloque de entrada un total de 64 vueltas, con 16 operaciones en cada una de estas 4 rondas los vectores se combinan de la siguiente forma:

1ª ronda F = (B AND C) OR (NOT B AND D)

2ª ronda G = (B AND D) OR (C AND NOT D)

3ª ronda H = (B XOR C XOR D)

4ª ronda I = (C XOR (B OR NOT D))

En cada función FGHI se usan las 16 palabras del bloque de 512 bytes (16 x 32 bits), bloques que van desde M0 hasta M15. Asimismo, en cada vuelta de las 64 vueltas se usan unas constantes Ki y desplazamientos s que vienen dados en unas tablas.

Función *hash* SHA-1

SHA-1, acrónimo de *Secure Hash Algorithm*, propuesto inicialmente en 1993 por la NSA y adoptado como estándar en 1995 por el NIST, sustituye al primer algoritmo de resumen SHA-0 en el que se habían detectado problemas. SHA-1 es muy similar a MD5, ya que también trata bloques de 512 bits de mensaje y sus operaciones lógicas son muy similares, si bien genera un resumen de 160 bits pues cuenta con 5

© Universidad Internacional de La Rioja (UNIR)

vectores de 32 bits, ABCDE, en vez de los 4 de MD5. En este caso, para cada bloque se realizan un total de 80 operaciones con palabras de 32 bits, organizadas en 4 rondas de 20 vueltas cada una.

Terminadas las primeras 80 vueltas, los vectores iniciales ABCDE habrán cambiado varias veces de valor, y serán los nuevos 5 vectores ABCDE los que se mezclarán con el segundo bloque de 512 bits de entrada. Esta acción se repite y se va encadenando con los siguientes bloques, hasta que el último valor de los vectores ABCDE es el resumen de 160 bits de todo el documento. Los vectores públicos en este caso son: A = 67452301, B = EFCDAB89, C = 98BADCFE, D = 10325476, E = C3D2E1F0.

Dependiendo de la ronda, los vectores se combinan de la siguiente forma:

1ª ronda F = (B AND C) OR (NOT B AND D)

2ª ronda G = (B XOR C XOR D)

3ª ronda H = (B AND C) OR (B AND D) OR (C AND D)

4ª ronda I = (B XOR C XOR D)

Función *hash* SHA-256 y SHA-3

En octubre de 2012, el NIST elige como ganador de SHA-3 al **algoritmo Keccak**, declarado como estándar mundial en agosto de 2015 por ese mismo organismo. Como desde que un algoritmo se declara como estándar mundial hasta que su uso puede considerarse masivo, pasan fácilmente 10 años (el AES solo comienza a usarse de manera masiva en comunicaciones SSL/TLS, diez años después de ser declarado nuevo estándar mundial de cifra simétrica), la NSA desarrolla en 2001 una familia de algoritmos *hash* conocida como SHA-2, que consiste en un conjunto de cuatro funciones *hash* de 224, 256, 384 y 512 bits. En estos dos últimos se trabaja con palabras de 64 bits. Su funcionamiento es muy similar al visto en MD5 y SHA-1.

© Universidad Internacional de La Rioja (UNIR)

En cuanto al algoritmo SHA-3, comentar que no tiene un esquema como los de la familia SHA y MD5, que todos eran muy similares en su concepción, sino que se trata de un sistema conocido como esponja en el que la entrada es una cadena de bits de cualquier longitud, que son absorbidos por esa esponja, y se obtiene una salida del tamaño de bits que se desee exprimiéndola. En su concepción, el algoritmo es bastante más complejo y se interpreta como un array en tres dimensiones, con movimientos simulares a los que se realizan sobre un cubo de Rubik.

A diferencia de MD5, definitivamente roto, el hash SHA-1 podría considerarse hoy en día medianamente seguro, si bien el febrero de 2017 sufrió un importante y detallado ataque, denominado **SHAttered.** Se ha usado en certificados digitales X.509 hasta 2016, pues a partir del 1 de enero de 2017 ya no se usa y en su lugar aparece SHA-256. Por ello, es recomendable usar en 2017 la familia SHA-2 o bien el nuevo estándar SHA-3.

© Universidad Internacional de La Rioja (UNIR)

6.4. Cifrado asimétrico

La cifra asimétrica, conocida también como cifra por clave, tiene sus inicios en noviembre de 1976 cuando dos investigadores de la Universidad de Stanford, **Whitfield Diffie y Martin Hellman**, proponen un nuevo esquema de cifra.

El nuevo sistema está basado en que la clave deja de ser única y, por tanto, secreta al estar compuesta por dos partes: una pública y conocida por todo el mundo; y la otra secreta, o privada, y conocida solo por su propietario. Mediante propiedades matemáticas de los cuerpos finitos, ambas claves son inversas entre sí y solo el dueño de esa clave puede a partir de una calcular la otra. A cualquier usuario que conozca la clave pública de otro usuario o del equipo en el sistema le será computacionalmente imposible descubrir la clave privada correspondiente, porque deberá enfrentarse a un problema de tipo NP (tiempo polinomial no determinista).

Cifra con la clave pública del destinatario. Si se cifra con la clave pública del destino, entonces solamente ese destinatario (y no otro) será capaz de descifrar el criptograma enviado, pues solo él o ella están en posesión de la clave privada. Esto es lo que en la práctica se conoce como intercambio seguro de clave. Resumiendo, cifrando con la clave pública del destino, se obtiene la **confidencialidad.**

Cifra con la clave privada del emisor. Si el emisor cifra con su clave privada, podrá descifrar ese criptograma y recuperar mensaje original todo aquel que conozca la clave pública del emisor. El mensaje en este caso no es un secreto porque lo pueden recuperar todos los usuarios que conozcan la clave pública del emisor. Sin embargo, quienes lo descifran pueden estar seguros de que ese mensaje es de ese emisor y no de otra persona, porque nadie excepto esa persona conoce esa clave privada. Si a ello unimos que lo que normalmente se cifra en el extremo emisor es el *hash* de un documento, entonces todo el proceso se conoce como **firma digital** y en el extremo receptor, además de autenticidad del emisor, se podrá comprobar la integridad del

© Universidad Internacional de La Rioja (UNIR)

mensaje debido a las propiedades de las funciones *hash*.

Aunque de forma genérica se hable de texto en claro o de mensaje M, lo cierto es que con estos sistemas asimétricos solo se cifran números. Más aún, números de unas pocas centenas de bits, como podrían ser una clave secreta o de sesión de un algoritmo simétrico como AES que se intercambia entre dos interlocutores, cliente y servidor, por ejemplo, o bien el valor de un *hash* SHA-256 que se va a firmar digitalmente. Para cifrar mensajes de mayor tamaño se utilizan los algoritmos de cifra simétrica que poseen una tasa de cifra mil veces superior.

Intercambio de clave de Diffie y Hellman

En 1976, Whitfield Diffie y Martin Hellman un algoritmo para intercambiar una clave secreta de manera computacionalmente segura, usando para ello funciones matemáticas de un solo sentido o unidireccionales. Esto significa que, si bien el sistema puede romperse matemáticamente, para los valores estándar de miles de bits que se usan en la práctica, el problema se vuelve intratable, tanto en recursos de cálculo como en tiempo. Y ahí radica su seguridad.

Diffie y Hellman usaron en su propuesta la función de un solo sentido conocida como **problema del logaritmo discreto.** Encontrar el resultado de la expresión $\alpha^{\beta} \bmod p = x$conocidos el generador *undefined*, xy el primo p, es sencilla y rápida, incluso para números grandes. Sin embargo, conociendo *undefined*, xy p, encontrar ahora el valor del exponente exponente $\beta = \log_{\alpha} x \bmod p$, se convierte en un problema de muy difícil solución para números grandes.

El algoritmo propuesto por Diffie y Hellman es el siguiente. Alicia y Bernardo, que se encuentran distantes, eligen un primo p y un número *undefined* dentro de ese primo, conocido como generador del primo. Ambos valores son públicos. Acto seguido, cada uno elige un número secreto que no conoce el otro. Así, Alicia elige el número secreto a y Bernardo elige el número secreto b. A continuación, realizan el siguiente protocolo:

© Universidad Internacional de La Rioja (UNIR)

- Alicia usa su clave secreta a, calcula $x_1 = \alpha^a \bmod p$y se lo envía a Bernardo.
- Bernardo usa su clave secreta b, calcula $x_2 = \alpha^b \bmod p$y se lo envía a Alicia.
- Bernardo recibe x_1 y con su clave secreta b calcula: $x_1^b = (\alpha^a)^b \bmod p$
- Alicia recibe x_2 y con su clave secreta a calcula: $x_2^a = \left(\alpha^b\right)^a \bmod p$
- La clave intercambiada entre Alicia y Bernardo será: $\alpha^{a \cdot b} \bmod p = \alpha^{b \cdot a} \bmod p = K$

Una vez terminado el protocolo, ambos comparten un mismo valor K que puede usarse como clave de sesión de forma segura. Cualquier intruso que conozca los datos públicos py *undefined*, así como los valores intercambiados x_1y x_2, para poder encontrar los valores secretos a de Alicia y b de Bernardo y, por tanto, poder calcular $\alpha^{a \cdot b} \bmod p$, se enfrentará al problema del logaritmo discreto, que para valores de p superiores a los mil bits se vuelve intratable. Nótese que el valor α debe ser una raíz primitiva o generador del primo p. Una raíz primitiva o generador dentro de un primo p, es aquel resto de ese primo que genera el conjunto completo de restos, excepto el 0, al hacer la operación de elevarlo a dichos restos reducido módulo p.

El algoritmo RSA

Es en febrero de 1978, es decir, poco más de un año después de la propuesta de Diffie y Hellman, cuando otros tres investigadores norteamericanos, Ron Rivest, Adi Shamir y Leonard Adleman, del Instituto Tecnológico de Massachusetts MIT, proponen un sistema de cifra que llevará las iniciales de sus apellidos. El algoritmo se patentará como **RSA.**

A diferencia del intercambio de clave de DH, RSA basa su fortaleza en la dificultad computacional de factorizar un número producto de dos primos grandes y encontrar tales factores primos. Este problema tiene una complejidad algorítmica similar al

© Universidad Internacional de La Rioja (UNIR)

problema del logaritmo discreto, siendo también inabordable para la capacidad mundial de cómputo en nuestros días cuando se trata de valores por encima de los dos mil bits.

Generando una clave RSA

El protocolo detallado para la generación de una clave RSA, pública y privada, es el siguiente:

- El usuario que va a generar las claves pública y privada (Alicia) elige dos primos p y q de valores iguales o superiores a 1 024 bits. Los valores de los primos p y q serán un secreto, solo conocido por Alicia.
- Alicia multiplican esos primos y obtiene el módulo de cifra $n = p \cdot q$ y hace público el módulo de cifra n.
- Ahora se calcula Indicador de Euler Φ de ese módulo n, que en el caso de dos primos multiplicados entre sí es $\Phi_n = (p-1)\cdot)$. Este valor es su cuerpo trampa, un número que solo conoce su dueño.
- A partir de ese valor trampa Φn se calculará ahora la clave pública e y la correspondiente clave privada d.
- Alicia escoge un valor de clave pública e que deberá estar en el intervalo 1 < e < Φn y ser coprimo de Φn, es decir, que mcd [e, Φn] = 1. En la práctica, esto no se hace, puesto que todo el mundo usará como clave pública e el número 4 de Fermat F4 = 224 + 1 = 216 + 1 = 65 537, un número primo que permite realizar las operaciones de cifra mucho más ágilmente.
- Ese valor e será la segunda parte de su clave pública, además del módulo n.
- Usando el Algoritmo Extendido de Euclides, cada usuario calculará su clave privada d en ese cuerpo trampa Φn. $d = \text{inv}\,(e, \Phi_n)$.
- Finalmente, Alicia tiene dos valores que forman su clave pública (n, e) y un valor

© Universidad Internacional de La Rioja (UNIR)

secreto que es su clave privada (d). Además del valor d, también guardará en secreto p y q, que le servirá para acelerar la operación de descifrado mediante el Teorema Chino del Resto.

A continuación, se muestra un **ejemplo** con números pequeños de este proceso:

- Alicia elige como primos los valores p = 7 y q = 17, con lo que su módulo de cifra será n = 7·17 = 119 y su cuerpo trampa Φn = (7 - 1)*(17 - 1) = 6*16 = 96.
- Elige como clave pública e = 5 ya que mcd (5, 96) = 1.
- Calcula su clave privada d = inv (5, 96) = 77.

Cifrando y descifrando con RSA

Si Alicia desea enviar el número secreto N (con confidencialidad) a Bernardo, realizará las siguientes operaciones. Como Alicia conoce la clave pública de Bernardo n y e, calcula Ne mod n = C, enviando a Bernardo este criptograma C. Dado que Bernardo es el único que conoce su clave privada d, solo él podrá realizar la siguiente operación: Cd mod n. Observa que Cd mod n = [Ne mod n]d mod n = [Ne]d mod n. Puesto que en la operación [Ne]d mod n los exponentes se anulan entre sí porque son inversos en Φn, en recepción Bernardo recuperará el mensaje secreto N enviado por Alicia dentro del cuerpo n.

Por ejemplo, conociendo que los valores públicos de la clave anterior de Bernardo son n = 143 y e = 7, Alicia le enviará el valor secreto N = 75. Como la clave privada de Bernardo era d = 103, se procede entonces así:

- Alicia calcula 757 mod 143 = 114, que envía a Bernardo.
- Bernardo calcula 114 103 mod 143 = 75, y recupera el secreto.

Firmando con RSA y comprobando la firma

© Universidad Internacional de La Rioja (UNIR)

Si Alicia desea enviar un mensaje H (posiblemente un *hash*) firmado a Bernardo, con su clave privada d va a realizar la siguiente operación Hd mod n = S, enviando en ese criptograma S su firma. Dado que Bernardo conocerá la clave pública de Alicia, n y e (y todos los que la conozcan) podrá realizar la siguiente operación: Se mod n = H.

Observa que Se mod n = [Hd mod n]e mod n = [Hd]e mod n. Como en la operación [Hd]e mod n los exponentes se anulan entre sí entregando el valor 1 como ya hemos visto, Bernardo recupera el mensaje H firmado por Alicia dentro del cuerpo n.

Como ya se ha comentado, en los sistemas de cifra asimétricos como es el caso de RSA, en la práctica solo se cifrarán números y además valores relativamente pequeños, de unas centenas de bits. Por tanto, una firma digital se realizará sobre un número, resultado de aplicar una función hash a un archivo ya de cualquier tamaño y tipo.

Los pasos a realizar en la firma digital con RSA a través de un *hash* serán:

- Alicia obtiene la función hash h(M) = H del archivo M que desea firmar.
- Sobre ese *hash* H, Alicia realizará la firma usando su clave privada d.
- Alicia calcula Hd mod n = S y le envía a Bernardo la dupla mensaje y firma (M, S).
- Bernardo recibe (M', S) pues no se sabe si recibe el mismo mensaje M o un mensaje distinto o falso M'.
- Bernardo realiza la siguiente operación con la clave pública de Alicia: Se mod n = [Hd mod n]e mod n = H.
- Es decir, recupera el valor H = h(M) que usó Alicia para su firma en el momento de emisión.
- Bernardo calcula la función *hash* h(M') del mensaje M' recibido, que puede ser el

© Universidad Internacional de La Rioja (UNIR)

verdadero mensaje M u otro distinto M'.

- Si el *hash* calculado en recepción coincide con el *hash* recuperado y firmado en emisión, se acepta la firma como correcta; en caso contrario se rechaza la firma.

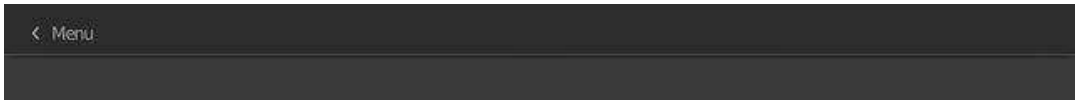
Si desea a firmar un documento M cuyo *hash* h(M) es el valor 46, se usará la clave privada d = 77 y su cuerpo de cifra n =119. En recepción, Bernardo, o quien conozca la clave pública e = 5 y n = 119 de Alicia, podrán comprobar dicha firma. Se procede en este caso así:

- Alicia calcula 4677 mod 119 = 65, que envía a Bernardo.
- Bernardo calcula 655 mod 119 = 46, y comprueba la firma.

Cifrando mensajes con RSA

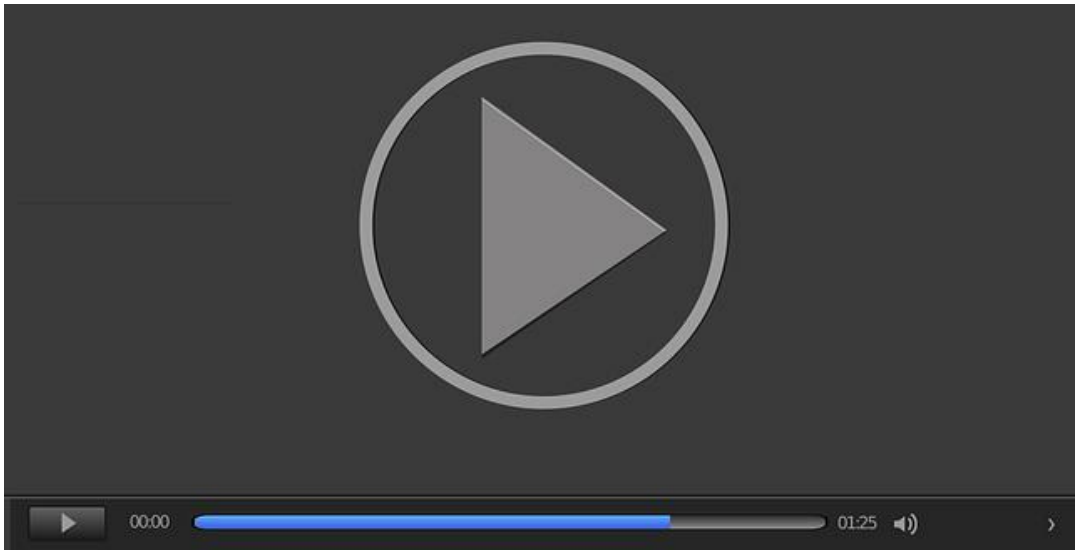
Puesto que los sistemas de cifra con clave pública tienen una velocidad de cifra en torno a los cientos de Kilobytes por segundo, muy lenta si la comparamos con la velocidad de los sistemas de cifra con clave secreta que está en los cientos de Megabytes por segundo (mil veces más rápidos), los primeros se usarán preferentemente para cifrar números de solo unas pocas centenas de bits. Por lo tanto, aunque podemos codificar el mensaje M con la codificación que se nos ocurra (código ASCII decimal, por ejemplo) para transformar esa información a un conjunto a números y usar luego RSA para cifrar esa cadena de valores, en la práctica no tiene sentido hacerlo por esa velocidad tan baja.

En el vídeo, *Cifrado asimétrico utilizando software educativo,* se muestra el uso de RSA para cifrar, firmar y verificar la firma con números pequeños y *software* educativo de modo que se facilite la comprensión del algoritmo en sí.



© Universidad Internacional de La Rioja (UNIR)

Ideas clave



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=63059790-4407-42fd-9003-af63012d4412

© Universidad Internacional de La Rioja (UNIR)

6.5. Referencias bibliográficas

Ramió, J. (2006). *Libro Electrónico de Seguridad Informática y Criptografía*, Versión 4.1. http://www.criptored.upm.es/guiateoria/gt_m001a.htm

© Universidad Internacional de La Rioja (UNIR)

Libro Electrónico de Seguridad Informática y Criptografía

Ramió, J. (2006). *Libro Electrónico de Seguridad Informática y Criptografía*, Versión 4.1. https://www.criptored.es/descarga/SegInfoCripPDFc_v41.zip

Este libro entra en detalle en cada uno de los algoritmos presentados en este tema, sin necesidad de ahondar en exceso en matemática compleja, sino en el significado y funcionamiento de estos.

© Universidad Internacional de La Rioja (UNIR)

Capítulos 5, 9 y 11 del libro Cryptography and Network Security: Principles and Practice

Stallings, W. (2019). *Cryptography and Network Security: Principles and Practice*, Eigth Edition. Pearson.

Los capítulos seleccionados de este libro amplían en bastante detalle los conceptos de criptografía aplicada a la seguridad en redes vistos en este tema.

© Universidad Internacional de La Rioja (UNIR)

Canal de YouTube <computerphile>

Canal de YouTube <computerphile> *Explicaciones detalladas y gráficas de algoritmos criptográficos*. https://www.youtube.com/user/computerphile

En este canal de YouTube se presentan paso a paso algunos de los algoritmos criptográficos estudiados en el presente tema. Algunos ejemplos de ello son:

AES https://www.youtube.com/watch?v=O4xNJsjtN6E

RSA https://www.youtube.com/watch?v=JD72Ry60eP4

© Universidad Internacional de La Rioja (UNIR)

Test

1. La diferencia entre sistemas de cifra simétrica y cifra asimétrica es que:

   A. La simétrica usa una clave para cifrar y otra distinta para descifrar.

   B. La asimétrica usa una clave para cifrar y otra distinta para descifrar.

   C. La simétrica cifra en bloques y la asimétrica lo hace en flujo.

   D. La simétrica cifra en flujo y la asimétrica lo hace en bloques.

2. El algoritmo AES:

   A. Es el sustituto de Rijndael como estándar de cifrado simétrico.

   B. Es el sustituto de IDEA como estándar de cifrado simétrico.

   C. Es el sustituto de RC4 como estándar de cifrado simétrico.

   D. Es el sustituto de DES como estándar de cifrado simétrico.

3. En el algoritmo AES, el tamaño estándar de la clave de cifra es:

   A. 64, 128 y 256 bits.

   B. 128, 206 y 256 bits.

   C. 128, 192 y 256 bits.

   D. 128, 256 y 512 bits.

4. Con un algoritmo asimétrico se suele:

   A. Cifrar mensajes de pocos bits.

   B. Cifrar mensajes del tamaño que sea requerido.

   C. Cifrar más rápido (en tasa de bits) que con los algoritmos de cifra simétrica.

   D. Los algoritmos asimétricos no permiten cifrar datos.

© Universidad Internacional de La Rioja (UNIR)

5. Las funciones *hash* permiten:

A. Autenticar a un usuario.

B. Demostrar la integridad de un documento o texto.

C. Firmar digitalmente un documento.

D. Son válidas las tres anteriores.

6. El hash MD5 realiza en cada bloque:

A. 64 vueltas.

B. 16 vueltas.

C. 80 vueltas.

D. 128 vueltas.

7. Indica cuál de estas afirmaciones es válida sobre el tamaño del *hash*:

A. MD5 124 bits y SHA-1 160 bits.

B. MD5 128 bits y SHA-1 164 bits.

C. MD5 128 bits y SHA-1 160 bits.

D. MD5 128 bits y SHA-1 168 bits.

8. Supuesto el sistema RSA con e = 5 y n = 35, el resultado de cifrar el número 3 será:

A. 5 ^ 35 mod 3.

B. 3 ^ 35 mod 5.

C. 5 ^ 3 mod 35.

D. 3 ^ 5 mod 35.

© Universidad Internacional de La Rioja (UNIR)

9. La seguridad de RSA se basa en:

A. La dificultad de factorización de números grandes compuestos por la multiplicación de 2 primos.

B. La dificultad de encontrar el logaritmo discreto en primos grandes.

C. La dificultad de factorización de números grandes primos.

D. Ninguna de las anteriores.

10. Los algoritmos de Diffie-Hellman y RSA:

A. Basan su seguridad en el problema del logaritmo discreto.

B. Son algoritmos de cifra asimétrica.

C. Son algoritmos de cifra simétrica.

D. Son algoritmos donde los interlocutores no guardan valores secretos.

© Universidad Internacional de La Rioja (UNIR)